

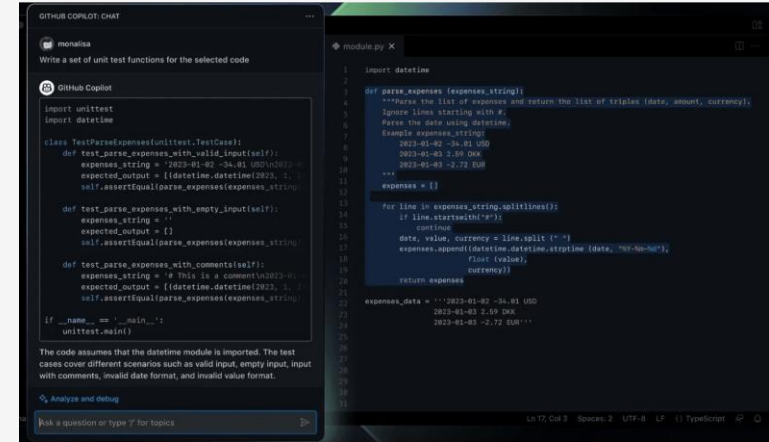
Generative Code Assistant and Context-Based Code Optimizer

Bridging Functionality and Mathematical Optimization

Raqib Hayat

NUM-BSCS-2022-40

Project Lead



The screenshot displays the GitHub Copilot interface. On the left, a chat window titled 'GITHUB COPILOT: CHAT' shows a conversation with 'monalisa'. The prompt is 'Write a set of unit test functions for the selected code'. The response shows a Python test class 'TestParseExpenses' with three methods: 'test_parse_expenses_with_valid_input', 'test_parse_expenses_with_empty_input', and 'test_parse_expenses_with_comments'. Below the code, a note states: 'The code assumes that the datetime module is imported. The test cases cover different scenarios such as valid input, empty input, input with comments, invalid date format, and invalid value format.' At the bottom, there is a button 'Analyze and debug' and a text input field 'Ask a question or type "!" for topics'. On the right, a code editor shows a file named 'module.py' with a function 'def parse_expenses(expenses_string):'. The function's docstring says: 'Parse the list of expenses and return the list of tuples (date, amount, currency). Ignore lines starting with #. Parse the date using datetime. Example expenses_string: 2023-01-02 -34.81 USD 2023-01-03 2.59 DKK 2023-01-03 -2.72 EUR'. The function body includes a loop that splits each line by whitespace, checks if it starts with '#', and then appends a tuple of (datetime.strptime(date, '%Y-%m-%d'), float(value), currency) to the 'expenses' list. The function returns 'expenses'.

Abu Bakar

NUM-BSCS-2022-41

Project Lead

The Problem: Code That Works, But Works Poorly

Current AI coding assistants focus on generating functional code, but ignore efficiency:

- **Functional but Unoptimized:** They give you working code, but it's inefficient—like a car that runs but guzzles gas.
- **Technical Debt:** Suboptimal code accumulates over time, making systems harder to maintain and upgrade.
- **Higher Cloud Costs:** Inefficient code consumes more computational resources, increasing cloud infrastructure expenses.
- **Performance Bottlenecks:** As systems scale, unoptimized code leads to slower execution and poor user experience.

The Missing Link

No tool systematically combines AI code generation with formal mathematical optimization principles and industry best practices.



The Solution: Context-Based Optimization

An intelligent chatbot combining **Generative AI** with a **Context-Based Optimization Engine** that applies mathematical formulas and industry best practices.

How It Works

1

Query

User submits code generation request or existing code for optimization.

2

RAG Retrieval

System retrieves optimal algorithmic patterns and optimization rules from knowledge base.

3

LLM + Math

LLM synthesizes code while applying mathematical optimization formulas.

4

Optimized Code

Delivers code with measurable improvements in time and space complexity.

Key Advantage

Code that is not just **functional**, but mathematically and practically **optimized**.



The Customer: Who Needs Optimized Code?

■ Junior & Mid-Level Developers

Need a tool to ensure their code meets senior-level performance standards and accelerate their learning curve.

Benefit: **Faster skill development** and confidence in code quality.

■ DevOps & Cloud Engineers

Need to reduce cloud costs by optimizing resource-intensive code segments and improving infrastructure efficiency.

Benefit: **Significant cost savings** on cloud infrastructure and computational resources.

■ Enterprise Software Companies

Companies where performance is critical (FinTech, Gaming, High-Frequency Trading) and technical debt is costly.

Benefit: **High-performance systems** that scale efficiently and maintain competitive advantage.



The Revenue Model: Monetizing Efficiency

Software as a Service (SaaS) Subscription Model

Free Tier

\$0/month

Basic code generation
Limited optimization suggestions
Community support

Pro Developer

\$29/month

Unlimited optimization
Full RAG knowledge base access
IDE integration
Priority support

Enterprise

Custom Pricing

Dedicated API access
On-premise deployment
Private codebase integration
Dedicated account manager



Key Metric

We sell **performance** and **cost savings**. Our value is directly tied to the efficiency gains we provide to customers.

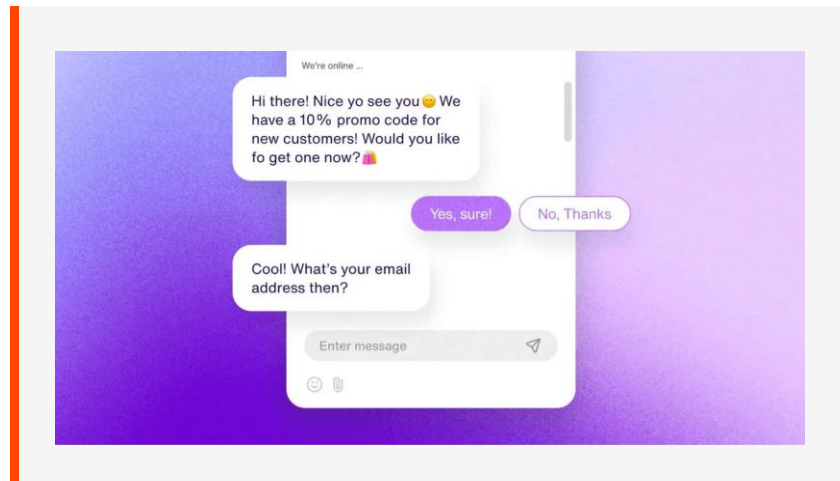
The Future of Optimized Software Development

Project Summary

Our project addresses the **critical need for efficient, mathematically-optimized code** in the age of AI-assisted development. We bridge the gap between tools that generate functional code and tools that ensure optimal performance.

Key Differentiator

We uniquely integrate **formal mathematical optimization** with **generative AI**, ensuring code is not just functional, but provably efficient.



Call to Action

We are ready to benchmark our solution against industry leaders to prove measurable performance gains.

Thank You!

Questions & Answers